



Computer Science Department

CipherSafe

Alex Biddle

Braeden Kinloch

Advisor: Dr. Elouni

Course: CSCI 403

June 8, 2026

Senior Capstone Project

Abstract

CipherSafe is a secure password management system developed to implement modern encryption and security principles in an easily accessible web application. The project emphasizes zero-knowledge practices, ensuring that user credentials are encrypted locally prior to storage or transmission, to prevent exposure of credentials to anybody. The system utilizes a PostgreSQL database and incorporates salted password hashing, session-based key management, and a modular backend design to enforce the confidentiality, availability, and integrity of all credentials. CipherSafe was implemented using web technologies that support encrypted communication between the client and the server, with a particular focus on minimizing persistent key storage and reducing potential attack surfaces. Comprehensive tests were administered to evaluate the security of the data. The results show that it is possible to securely manage encryption keys in a web application without sacrificing ease of use. In the future, CipherSafe could be improved by adding features such as device synchronization, browser extension integration, biometric authentication support, and multi-factor authentication support.

1 Introduction

A client-based password manager application allows users to store and access their usernames, emails, and passwords for all their accounts. In this project, we will implement CipherSafe, a modern online web-based password manager that uses modern security policies to protect users' information and ensure its integrity. Two servers will be used: one will host the client's view of passwords, and the other will serve as a private API server where all users' data will be stored within a database. We will implement zero-knowledge, end-to-end encryption, least privilege, and key separation principles. These will ensure confidentiality and reduce attack risks and surfaces. Modern networking protocols and authentication practices will be used to ensure that only authorized entities handle or view confidential data on our servers.

2 Background

Password managers have become an essential asset for people to take advantage of in today's modern digital age as organizations and individuals aim to secure their passwords and other sensitive credentials. The top priority of password managers is the confidentiality, integrity, and availability of usernames and passwords. Password managers successfully carry out this process by protecting users' data with encryption and hashing, to prevent malicious attacks such as password brute force, phishing, session hijacking, and server compromises [3].

There are many types of password managers, with some being local and isolated to one machine, while others are online web applications that handle all their data through a network or cloud. Figure 1 presents the different password managers and the various attributes and services that different password managers offer. There are many types of password managers, with some being local and isolated to one machine, while others are online web applications that handle all their data through a network or cloud.

Password managers have faced many persistent problems and challenges throughout their history. One such problem is the use of a weak master password, as a weak master password can

System		Supports generation	Supports autofill	Cloud sync for extension settings	Cloud sync for vault	CLI support	Supports MFA	Lockable Vault	Login on separate tab or app	Has assessment tool	Clears clipboard	Open Source
App	KeePassX	●	○	○	○	○	○	●	●	○	●	●
	KeePassXC	●	●	○	○	●	○	○	○	○	●	●
Extension	1Password X	●	●	○	●	●	●	○	●	○	○	○
	Bitwarden	●	●	○	●	●	○	○	○	○	○	●
	Dashlane	●	●	○	●	○	○	○	●	○	○	○
	LastPass	●	●	○	●	●	○	○	●	○	○	○
	RoboForm	●	●	○	●	○	○	○	●	○	○	○
Browser	Chrome	○	●	○	●	○	○	○	○	○	○	○
	Edge	○	●	○	●	○	○	○	○	○	○	○
	Firefox	○	●	○	●	○	○	○	○	○	○	○
	IE	○	●	○	●	○	○	○	○	○	○	○
	Opera	○	●	○	●	○	○	○	○	○	○	○
	Safari	○	●	○	●	○	○	○	○	○	○	○

Figure 1: Analyzed Password Manger Figure - [2]

lead to brute-force attacks, dictionary attacks, password spray attacks, and many more attacks that can compromise the security of a system. Weak master passwords and passwords, in general, have led to serious breaches of security and confidential information across corporate, government, and personal accounts. These breaches have caused billions of dollars in financial losses and reputational damage over the years. Password managers can be compromised if the companies behind them fail to implement strong security measures, such as robust encryption, secure networks, and effective intrusion detection systems. [2]. The shortcomings of existing password managers show that a secure solution must incorporate modern encryption techniques and adhere to strict security policies. In addition to following cybersecurity best practices, we must proactively address weaknesses in password management and enhance threat detection within the system.

Encryption and hashing are widely adopted security practices that protect the confidentiality of data while at rest and in transit. By definition, encryption is the process of converting data into an

unreadable form, so it cannot be viewed. This process is completed by different algorithms such as AES, RSA, and other standards of encryption. Encrypted data can only be viewed once the reversal or decryption of the data has occurred. Hashing is the process of transforming data into a fixed-length string called a hash using a cryptographic algorithm. Unlike encryption, hashing is one-way and cannot be reversed, which makes it ideal for securely storing sensitive information such as passwords.

Symmetric encryption is an encryption method that uses one key to protect the data. The shared key is used to encrypt and decrypt the sensitive data. Symmetric encryption is a method of encryption for storing information locally because it is a simple and fast way of encrypting and decrypting information, but it suffers when sending sensitive information over a network because the key is the same, making key exchange not secure. If the key was intercepted, anybody could decrypt the information easily.

Asymmetric encryption differs from symmetric encryption as it uses a pair of keys, one public key and one private key, to encrypt securely and decrypt information. The public key is used to encrypt the data, which anybody can use. For decryption, only the private key can be used. Asymmetric encryption is great for sending encrypted information over a network because key exchange is secure. Asymmetric encryption suffers when large sets of data are used because it is slow, complex, and can be computationally expensive.

CipherSafe uses both symmetric and asymmetric encryption practices to ensure data is securely transmitted into storage and allows users to access stored credentials quickly. Symmetric encryption will be used for encrypting data locally, while asymmetric encryption will be used to encrypt data in transit.

3 Design and Implementation

3.1 Client Website

The client website of CipherSafe implements a sleek and professional user interface that users interact with to store and view their passwords and other credentials.

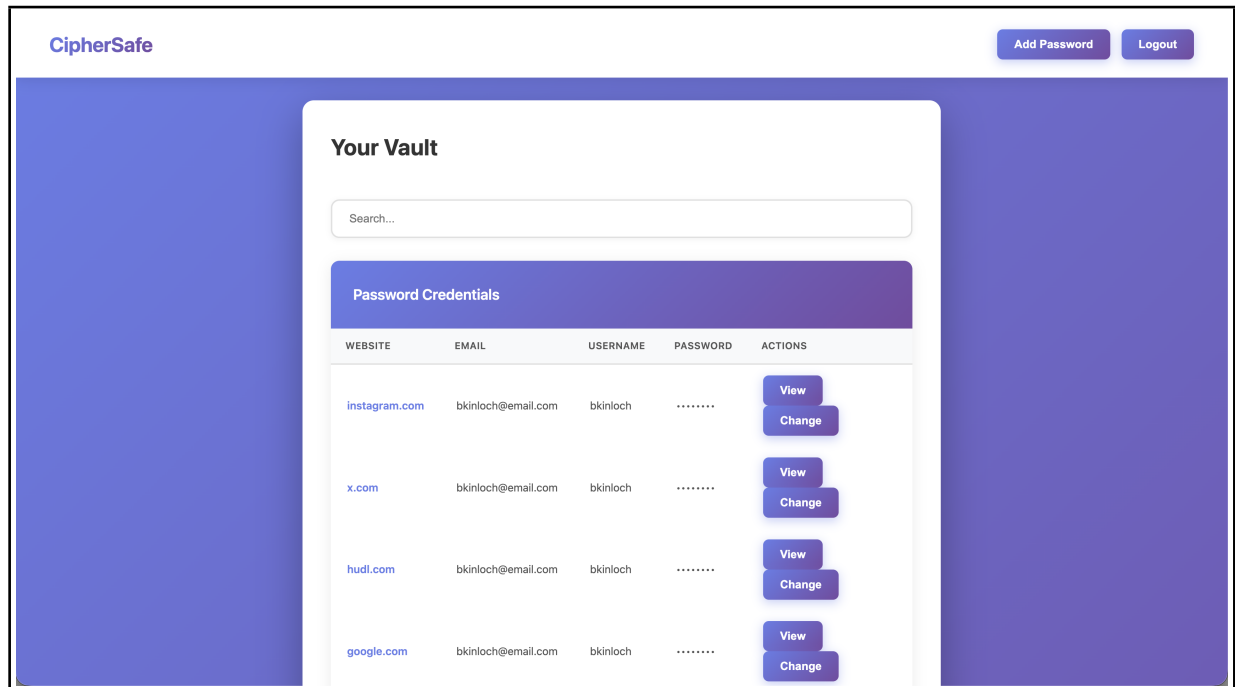


Figure 2: The Interface of CipherSafe

The website uses HTML and JavaScript to form the components that create the user's view and use of CipherSafe. The web pages consist of a login page, a page to register, a page to add new credentials, and a page to view the user's vault. The login page takes the user's username or email and password to authenticate them into their vault. The register page allows users to create an account for CipherSafe. By entering a username or email and a password, a new user account will be created to store various credentials.

The vault page is the main page when logged in. The page contains all the users' websites for which they have credentials stored. When a user clicks on the website they wish to view, their cre-

credentials will be retrieved and shown to the user so they can log in to the desired website. The add page allows users to add credentials to a given website. The page will ask the user for their username, email, and password. The credentials will be encrypted and then securely sent and stored in the database. The client interacts with the Back-End API server using the four-way handshake to securely and safely transmit all of the data into the database.

3.2 Back-End API Server

The back-end API server is responsible for running all of the main operations of CipherSafe; it runs using a Flask Python server. The server is responsible for handling authentication, session management, encryption, and all interactions with the PostgreSQL database. The server was designed with strict isolation of sensitive operations, implementing modern cryptography, and a top-down approach with modular code.

To ensure the integrity of a user's master password, CipherSafe employs a two-stage Argon2id hashing routine. During registration, the client generates a unique client salt, which is used to hash the user's master password before it leaves the client website. Once the hashed master password reaches the server, a separate server salt is created and used to hash the client's hashed password again, which is then stored and used for verification when a user attempts to log in. When a user does attempt to log in, the server first sends the client salt to the user in the "hello server" step of the four-way handshake protocol. At no point does the plain text master password have any interaction with the server, ensuring a true zero-knowledge authentication model.

3.3 Four-Way Handshake Protocol

CipherSafe uses a four-way handshake protocol to authenticate client and server communication to allow for secure encryption key exchange. The protocol verifies message integrity, exchanges

cryptographic keys, and establishes an encrypted session for secure data transmission. The handshake operates entirely over TLS 1.3 and HTTPS 3.0, which guarantees confidentiality, integrity, and availability for all communications between the client and the backend API python server[1].

All sensitive data that is exchanged between the server and client is secured under the encryption of TLS 1.3 and HTTPS 3.0, but also uses Argon2id, AES-256-GCM, RSA-OAEP, RSA-PSS, and BLAKE2b-256 for encrypting and hashing to provide secure transmission, storage, digital signatures, and checksum of all data providing confidentiality, integrity, and availability across the transport and application layers. This provides an extra layer of protection to prevent man-in-the-middle, replay, key-substitution, and other attacks. There are four steps to the handshake protocol: first, the Client Hello, the Server Hello, the Client Encrypted Request, and the Server Encrypted Response. Each step in the protocol uses HTTPS packets to communicate, where data is formatted in JSON.

3.4 Four-way Handshake Steps

- **Client Hello**

The client hello step in the protocol establishes the first connection with the server. The client sends the server the protocol version, request type, username, the client's RSA public key, timestamp, and the client's checksum to establish a connection with the server and to announce what type of request the client will be initiating.

- **Server Hello**

The server hello step in the protocol responds to the client and establishes a user session in its session handler. The server will send back the protocol version, response status, username, timestamp, server checksum, server AES session key, server RSA public key, the server key identifier, the server key expiry, the server's digital signature, and the server's cipher text to

establish the connection between the server in the client. The server includes a cipher text field. In the case of a user requesting login, it will respond with the client salt value, which is encrypted with the client's public RSA key, along with the server AES session key, which is also encrypted with the client's public RSA key.

- **Client Encrypted Request**

The client encrypted request step in the protocol is where the Client will send an encrypted request to the server, where the data is encrypted with the server's AES session key. This request includes the protocol version, request type, username, timestamp, client checksum, server AES session key, server key identifier, and the client cipher text. The client cipher text will contain the user's hashed password as well as the data pertaining to the user's request type.

- **Server Encrypted Response**

The server encrypted request step in the protocol occurs after the client encrypted request. The server will respond with an encrypted response. This response includes the protocol version, response status, username, timestamp, server checksum, server cipher text, and the server digital signal. Based on the user's request type such as login, sign up for a new account, fetch all of the users accounts, fetch a password, add an account to a user's vault, update the information for an account, delete an account, update the users master password for the vault, and logout the server will execute the proper commands to retrieve or change data in the database and format the server cipher text with the proper information. The server will then send back a response status such as success, invalid account, authentication failed, user account already exists, invalid checksum, or failure.

- **Error Handling**

All errors that occur during the execution of the server's services will propagate up to the server error handler, where the appropriate HTTP status and error codes will be sent back to

the client. The various HTTP codes and error codes are each detailed with proper responses for the client to view, and all errors are logged on the back-end API server

3.5 Database

CipherSafe uses a PostgreSQL database where all the users' credentials are securely stored with encryption, and uses authentication before any access is allowed. The database was designed to be modular, isolate sensitive information, minimize data exposure, and enforce a strict zero-knowledge architecture to keep sensitive data separated from the general application. No unencrypted data, credentials, or passwords are ever stored in the database at any time [4].

At the core of the database design, there are two authenticated tables: the salt storage table and the master user table. The salt storage table stores the username, and an associated user identifier using UUID, client salts, and server salts which are required for the two-stage Argon2id password hashing process. The master user table stores each user's username, the server's hashed master password, and the user identifier. We decided to separate the salt table and the hashed master password to enhance the security of the database by ensuring that even if one table is exposed, an attacker could not reconstruct a user's master password without compromising both tables.

When a new user registers, the server assigns them a UUID. The UUID serves as the value to reference finding user salts and the user's associated table holding their vault data. CipherSafe creates a dedicated user table for each user, with the table able to be referenced using the user's UUID. These user tables store the encrypted account entries. The data stored includes their website names, usernames, emails, and passwords, which are encrypted using AES.

By combining per-user data table isolation and using non-guessable UUID references with separation of salts and master user table, CipherSafe's database architecture reinforces CipherSafe's

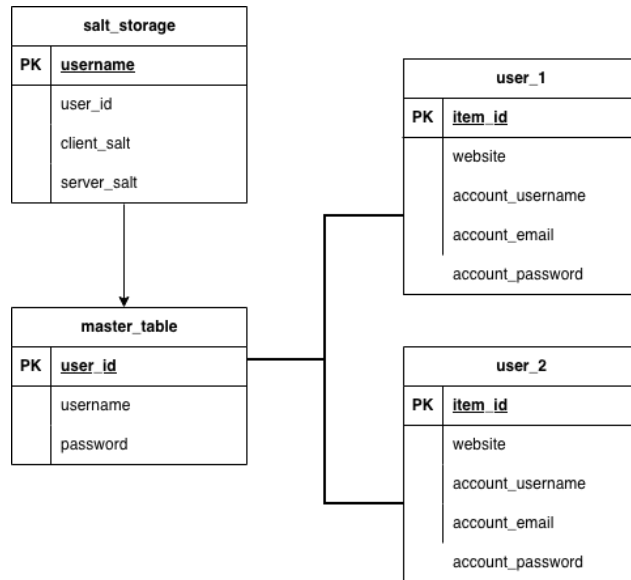


Figure 3: Database Schema

mission of achieving confidentiality, maintaining data integrity, and preserving the end-to-end encryption model.

4 Security Practices and Error Handling

All client and server messages use a cryptographic BLAKE2b-256 checksum with predetermined values from the packet to compute the checksum. This checksum ensures the integrity of the message after transit, as checksum validation is performed before decryption or signature verification to detect any tampering.

For mutual authentication to be achieved between the client and server a 2048-bit RSA key pair is used for asymmetric encryption, where as RSA-OAEP is used to encrypt the server's 256-bit AES session key for secure transport to the client during the Server Hello stage of the four-way handshake. RSA-PSS is used to digitally sign all server response packets, this digital signature allows the clients to verify the authenticity of the packet. The server automatically rotates its RSA

key pair every 24 hours, where it stores the private key on disk with AES-based file encryption with Flask and the restricted file system permissions so that the file is secure. The public key is dynamically derived as needed for sharing with the client. A AES-256-GCM session key is used for authenticated encryption for all secure request and responses in the four-way handshake. Each encrypted ciphertext block will include a unique 12-byte nonce and a 16-byte authentication tag that will be used for authentication and to prevent replay attacks.

User's master passwords are hashed using Argon2id and unique salts. The clients master password is first hashed using a client-side unique salt and then hashed again using a server-side unique salt. This dual-hashing design prevents raw password exposure, and provides a secure verification process for authentication.

Each packet sent in the four-way handshake comes with a timestamp field which are used to validate the packet to ensure a strict ± 5 second drift window to prevent any replay message attacks. When each user logs in, a user session is created, which expires automatically after 15 minutes. Expired sessions are securely destroyed, and standardized logout packets are generated by a dedicated cleanup thread running every 60 seconds on the back-end API server.

There are various security controls to protect data. Rate limiting is in place to 3 maximum login attempts per minute per IP address to protect against brute-force attacks and audit logging to keep track of the metadata, request types, outcomes, key identifiers, and checksums to keep track of malicious behavior. These requests go through the server to access the database to retrieve and store information in the user's vault. All of these parts come together in harmony to create a secure protocol infrastructure that users can trust to share passwords over the internet.

5 Conclusion and Future Work

In this work, the secure password manager CipherSafe has been created. It allows users to easily store and access their credentials to various websites with the peace of mind that their data is secure and cannot be accessed by anyone else. This is accomplished by implementing modern encryption algorithms such as AES, TLS, RSA, and Argon2id. The backend server communicates with the database to securely enter and retrieve credentials. Due to these factors, confidentiality, integrity, and availability of data are ensured.

Future additions to CipherSafe could include biometric authentication, multi-factor authentication, and a browser extension. Adding biometric authentication will exponentially increase security. Instead of just having a secure password to authenticate a user to our service, their own biological makeup will be the key to authentication. This includes fingerprint and facial scanning on users' devices. This will make sure that only the user themselves can authenticate into CipherSafe, even if somebody has their password. Multi-factor authentication (MFA) also decreases the chance that threat actors can steal users' data. This will be done by using MFA services such as Duo or Google Authenticator to make sure a user can only be authenticated with their permission and their device or hardware key. Adding a browser extension will increase the ease of use of CipherSafe. It will allow users to automatically save their passwords when they make a new account or update their password for a given website. It will also auto-fill their passwords when they need to log on to a website. Instead of needing to access CipherSafe's website and log in every time they need their password, it will be a seamless drop-down experience when logging into any website.

References

- [1] Harshit Dubey, Sudhakar Kumar, and Anureet Chhabra. 2022. Cyber Security Model to Secure Data Transmission using Cloud Cryptography. *CSIM 2* (2022). https://insights2techinfo.com/wp-content/uploads/2022/11/Cyber-Security-Model-to-Secure-Data-Transmission-using-Cloud-Cryptography_final_2.pdf
- [2] Sean Oesch and Scott Ruoti. 2020. That Was Then, This Is Now: A Security Evaluation of Password Generation, Storage, and Autofill in Browser-Based Password Managers. *9th USENIX Security Symposium* (Aug. 2020), 2165–2179. https://www.usenix.org/system/files/sec20-oesch_0.pdf
- [3] Chaitanya Rahalkar and Dhaval Gujar. 2019. A Secure Password Manager. *International Journal of Computer Applications* 78, 144 (Aug. 2019), 5–9. https://web.archive.org/web/20200212082140id_/https://pdfs.semanticscholar.org/53d3/6aeef69ad2565179101f2df971e34f156d87.pdf
- [4] Moaaz Bassam Yanes. 2023. Development of a secure, role-based password manager. (May 2023). <https://ntnuopen.ntnu.no/ntnu-xmlui/bitstream/handle/11250/3078086/no.ntnu%3ainspera%3a146721987%3a149459175.pdf?sequence=1&isAllowed=y>